

WALKTHROUGH LAB — WEEK 5

C Standard Library: Hands-On Lab

Estimated time: 2–3 hours

Prerequisites: gcc installed, Linux/macOS terminal or WSL on Windows

Setup

Create a directory for today's lab:

```
bashmkdir week5_lab
```

```
cd week5_lab
```

You will create one .c file per exercise. Compile each with:

```
bashgcc -Wall -o output_name source_file.c
```

PART 1: printf and Format Strings

Exercise 1.1 — Basic printf

Step 1: Create ex1_printf.c:

```
c#include <stdio.h>
```

```
int main(void) {
```

```
int age = 21;
```

```
float gpa = 3.75;
```

```
char grade = 'A';
```

```
char name[] = "Alice";
```

```
// Print each variable using the correct format specifier
```

```
printf("Name:  %s\n", name);
```

```
printf("Age:   %d\n", age);
```

```
printf("Grade: %c\n", grade);
```

```
printf("GPA:   %f\n", gpa);
```

```
printf("GPA (2 decimal places): %.2f\n", gpa);
```

```
// printf returns the number of characters written
```

```
int chars = printf("This line has how many characters?\n");
```

```
printf("Answer: %d characters were printed\n", chars);
```

```
return 0;
```

```
}
```

Step 2: Compile and run:

```
bashgcc -Wall -o ex1 ex1_printf.c
./ex1
```

Step 3: Expected output — observe how %.2f limits decimal places. Note the return value of printf.

Step 4: Now deliberately create the format string vulnerability. Add this at the end of main:

```
c char dangerous[] = "Hello %d world";
// NEVER DO THIS IN REAL CODE:
printf(dangerous); // val is treated as format string!
```

Compile with gcc -Wall — notice the warning. Run it and observe the garbage output.

Step 5: Fix it:

```
c printf("%s\n", dangerous); // ALWAYS use an explicit format string
```

Exercise 1.2 — scanf Input

Step 1: Create ex2_scanf.c:

```
c#include <stdio.h>
```

```
int main(void) {
    int num;
    float fnum;
    char word[50];
```

```
    printf("Enter an integer: ");
    int result = scanf("%d", &num);
    printf("scanf returned: %d, you entered: %d\n", result, num);

    printf("Enter a float: ");
    result = scanf("%f", &fnum);
    printf("scanf returned: %d, you entered: %.2f\n", result, fnum);

    printf("Enter a word (no spaces): ");
```

```
result = scanf("%s", word);
printf("scanf returned: %d, you entered: %s\n", result, word);

return 0;
```

```
}
```

Step 2: Compile and run. Try entering the right types. Then run again and enter a letter when it asks for an integer — observe what scanf returns (0 = nothing matched).

Step 3: Notice the newline issue. After scanf("%d", &num), the newline is still in the buffer. Add this after the first scanf:

```
c // Consume the leftover newline
int c;
while ((c = getchar()) != '\n' && c != EOF);
```

Exercise 1.3 — getchar Loop

Step 1: Create ex3_getchar.c:

```
c#include <stdio.h>
```

```
int main(void) {
int c;
int count = 0;
```

```
printf("Type a line and press Enter:\n");

while ((c = getchar()) != '\n' && c != EOF) {
    printf("  char[%d] = '%c' (ASCII %d)\n", count, c, c);
    count++;
}

printf("Total characters (excluding newline): %d\n", count);
return 0;
```

```
}
```

Step 2: Compile, run, type “hello world”. Observe each character printed with its ASCII value.

Step 3: Modify the program to count vowels:

c // Add inside the while loop:

```
if (c=='a' || c=='e' || c=='i' || c=='o' || c=='u' ||  
c=='A' || c=='E' || c=='I' || c=='O' || c=='U') {  
    printf(" ^ vowel!\n");  
}
```

PART 2: File I/O

Exercise 2.1 — Writing Text with fprintf

Step 1: Create ex4_write_text.c:

```
c#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main(void) {
```

```
    FILE *f;
```

```
    // Open file for writing  
    if ((f = fopen("students.txt", "w")) == NULL) {  
        perror("Error opening file");  
        return 1;  
    }
```

```
    // Write header  
    fprintf(f, "Name,Year,Group\n");
```

```
    // Write student records  
    fprintf(f, "%s,%d,%d\n", "Alice", 3, 1);  
    fprintf(f, "%s,%d,%d\n", "Bob", 3, 2);  
    fprintf(f, "%s,%d,%d\n", "Carol", 2, 1);  
    fprintf(f, "%s,%d,%d\n", "Dave", 2, 2);
```

```
    fclose(f);
```

```
    printf("Written to students.txt\n");  
    printf("Check with: cat students.txt\n");
```

```
return 0;
```

```
}
```

Step 2: Compile, run, then check the output:

```
bashgcc -Wall -o ex4 ex4_write_text.c
```

```
./ex4
```

```
cat students.txt
```

Step 3: Run the program again. What happens to the existing file? (It gets overwritten because we used “w” mode.)

Now change “w” to “a” and run again. How does the file change?

Exercise 2.2 — Reading Text with fscanf

Step 1: Create ex5_read_text.c:

```
c#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main(void) {
```

```
FILE *f;
```

```
char name[50];
```

```
int year, group;
```

```
if ((f = fopen("students.txt", "r")) == NULL) {  
    perror("Cannot open students.txt");  
    printf("Run ex4 first to create the file\n");  
    return 1;  
}
```

```
// Skip the header line
```

```
char header[100];
```

```
fgets(header, sizeof(header), f);
```

```
printf("Header: %s", header);
```

```
// Read all student records
```

```
int count = 0;
```

```

printf("\nStudents:\n");
while (fscanf(f, "%49[^\n],%d,%d\n", name, &year, &group) == 3) {
    printf("  Name: %-10s  Year: %d  Group: %d\n", name, year, group);
    count++;
}
printf("\nTotal: %d students\n", count);

fclose(f);
return 0;

```

```

}

```

Step 2: Compile and run (make sure students.txt exists from ex4):

```
bashgcc -Wall -o ex5 ex5_read_text.c
```

```
./ex5
```

Step 3: Notice %-10s in the format — the - left-aligns the string in a field of width 10. Experiment with different widths.

Exercise 2.3 — Binary I/O with fwrite/fread

Step 1: Create ex6_binary.c:

```
c#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
typedef struct {
```

```
char name[20];
```

```
int score;
```

```
float percentage;
```

```
} Record;
```

```
// Write records to binary file
```

```
void write_records(const char *filename) {
```

```
Record data[] = {
```

```
{“Alice”, 85, 91.2f},
```

```
{“Bob”, 72, 77.4f},
```

```
{“Carol”, 95, 98.6f}
```

```
};
```

```
int n = 3;
```

```
FILE *f = fopen(filename, "wb");
if (!f) { perror("write fopen"); return; }

// Write the count first
fwrite(&n, sizeof(int), 1, f);
// Write all records
size_t written = fwrite(data, sizeof(Record), n, f);
printf("Wrote %zu records (%zu bytes each)\n", written, sizeof(Record));

fclose(f);
```

```
}
```

```
// Read records from binary file
```

```
void read_records(const char *filename) {
```

```
FILE *f = fopen(filename, "rb");
```

```
if (!f) { perror("read fopen"); return; }
```

```
// Read the count
int n;
fread(&n, sizeof(int), 1, f);
printf("File contains %d records\n", n);

// Allocate and read
Record *data = malloc(n * sizeof(Record));
if (!data) { perror("malloc"); fclose(f); return; }

size_t numread = fread(data, sizeof(Record), n, f);
printf("Read %zu records:\n", numread);
```

```

for (int i = 0; i < (int)numread; i++) {
    printf("  %-10s score=%d  (%.1f%%)\n",
           data[i].name, data[i].score, data[i].percentage);
}

free(data);
fclose(f);

```

```

}

```

```

int main(void) {
    const char *file = "records.bin";
    write_records(file);
    printf("\nNow reading back:\n");
    read_records(file);

```

```

// Try to cat it – mostly garbage
printf("\nFile is binary – 'cat records.bin' will show garbage\n");
return 0;

```

```

}

```

Step 2: Compile and run:

```
bashgcc -Wall -o ex6 ex6_binary.c
```

```
./ex6
```

```
xxd records.bin | head # inspect the raw bytes
```

Step 3: Look at the hex dump. Can you identify the names? The scores? What byte order are the integers in?

Exercise 2.4 — fseek: Random Access

Step 1: Create ex7_fseek.c:

```
c#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct {
```

```
char name[20];
```



```
int score;
float percentage;
} Record;
```

```
int main(void) {
// Read just the Nth record from the binary file
FILE *f = fopen("records.bin", "rb");
if (!f) {
printf("Run ex6 first\n");
return 1;
}
```

```
// Skip the count integer at the beginning
int n;
fread(&n, sizeof(int), 1, f);
printf("File has %d records\n", n);

// Jump directly to record index 2 (the third one)
int target = 2;
if (target >= n) {
    printf("Record %d doesn't exist\n", target);
    fclose(f);
    return 1;
}

// Calculate offset: after the count int, skip 'target' records
fseek(f, target * sizeof(Record), SEEK_CUR);

Record r;
fread(&r, sizeof(Record), 1, f);
printf("Record %d: %s score=%d (%.1f%%)\n",
        target, r.name, r.score, r.percentage);

// Go to beginning of records and read the first one
fseek(f, sizeof(int), SEEK_SET); // skip the count, go to first record
```

```
fread(&r, sizeof(Record), 1, f);  
printf("Record 0: %s score=%d (%.1f%%)\n",  
       r.name, r.score, r.percentage);  
  
fclose(f);  
return 0;
```

```
}
```

Step 2: Compile and run. Observe direct random access to any record.